

The Signature of Scheming: Cross-Organism Interpretability of Strategic Misrepresentation

Project page: [SPAR project](#)

Mentors: David Williams-King, Linh Le, and Hong Kiat Tan

Applicant name: Raghavendra Kaushik Archak

Question 1 - Coding Automation Timeline

In your opinion, how long will it be until nearly all coding tasks can be automated by AI? Why do you say this timeline? Optional reading: [Measuring AI Ability to Complete Long Software Tasks](#) (200 words)

Current models are already good at many coding tasks that have a well-specified goal. Developers, even at frontier labs, are using them extensively [1]. If we use eight-hour tasks with an 80% success rate - roughly the duration of a software engineer's workday - as a proxy, my extrapolation of the METR task-horizon trend suggests somewhere around mid-2027 [2]. SWE-Bench Pro also suggests rapid progress. OpenAI reported one frontier model scoring 80.3%, although evaluation conditions differ; it also warns that many benchmark tasks may be broken [3].

I think it is plausible that by 2030 we could see swarms of agents autonomously working on codebases while humans mostly nudge their direction, specify needs, and review outputs [4].

Some factors could delay progress, such as a deliberate slowdown in response to safety risks, as proposed in AI 2040 [5], or constraints in power availability and chip manufacturing [6]. On the other hand, it could happen earlier if a lab makes a practical breakthrough in robust continual learning, allowing agents to accumulate experience across tasks [7].

Question 2 - Naive J-Space Sandbagging Detection

What would be a simple and naive way to detect sandbagging using the new J-space interpretability technique? [Verbalizable Representations Form a Global Workspace in Language Models](#) (200 words)

The J-lens lets us look at some intermediate concepts involved in the computation that produces the final answer. We can compare the top-k tokens in the J-space of middle-to-late layers under two conditions:

- The same sandbagging-tuned model under sandbagging and non-sandbagging conditions.
- A control model and a sandbagging-tuned model on the same prompts.

In either or both comparisons, the tokens obtained from the J-lens readout might differ. Naively, one might expect tokens such as "cheat," "deceive," "fake," "trick," "weak," or "eval" to appear in the J-space when the model is trying to sandbag. If we find such features, we could causally manipulate them by reducing their strength and testing whether sandbagging is reduced. However, relevant tokens can appear in the J-lens without causally controlling the behavior. For example, the paper found that a Spanish-to-French swap did not affect automatic continuation or anomaly

detection [8]. Therefore, sufficiently automatic sandbagging might evade this method.

If the standard single-token J-lens method fails, we could also try the multi-token methods suggested in the original paper: the template lens and oracle lens [8].

Question 3 - Reversing Cedar's Sandbagging

In "[Auditing Games for Sandbagging](#)", what was the strongest type of sandbagging model organism? Do you think it would be possible to reverse its sandbagging? (200 words)

The strongest sandbagging model was Cedar, which was intended for broad sandbagging where black-box methods failed. The model organism would act dumb when evaluated without a password. By removing sandbagging, we mean that the model performs with full evaluation performance even when there is no password. One can therefore fine-tune it on one correct on-distribution demonstration without a password, where the response is taken from a stronger model. The paper found that this fine-tuning elicited Cedar's full evaluation performance without the password, although it did not establish whether the underlying password-lock mechanism was permanently erased [9].

More speculatively, if we could find a steering vector or a single direction associated with sandbagging, similar to the direction found for refusal [10], we could ablate the sandbagging direction or add an opposing steering vector and test whether this reduces sandbagging at inference time.

References

- [1] Anthropic, "How AI Is Transforming Work at Anthropic." [Anthropic](#)
- [2] METR, "Measuring AI Ability to Complete Long Software Tasks." [METR](#)
- [3] OpenAI, "Separating Signal from Noise in Coding Evaluations." [OpenAI](#)
- [4] Alexander, "Introducing AI 2027." [Astral Codex Ten](#)
- [5] "AI 2040: Plan A." [AI 2040](#)
- [6] Epoch AI, "Can AI Scaling Continue Through 2030?" [Epoch AI](#)
- [7] Joshi, Chowdhury, and Uysal, "SWE-Bench-CL: Continual Learning for Coding Agents." [arXiv](#)
- [8] Lindsey et al., "Verbalizable Representations Form a Global Workspace in Language Models." [Transformer Circuits](#)
- [9] Taylor et al., "Auditing Games for Sandbagging." [arXiv](#)
- [10] Arditi et al., "Refusal in Language Models Is Mediated by a Single Direction." [arXiv](#)